Cooperative localisation without anchors

Topology, observability and intermittent GPS for a GPS-denied drone swarm — working notes, 20 October 2026

1. The scenario, and why it is well posed

Drones operate in open terrain with no surveyed infrastructure, so there are no true anchors. Ranging between drones is available continuously; GPS is available intermittently and with metre-level noise. The question an examiner will ask first is how the network can be localised at all without a fixed reference. It is worth answering that explicitly and early.

Range measurements between nodes determine the **shape** of the formation but not its placement. In the plane, a set of inter-node ranges fixes the configuration only up to a global rigid transform: two translation degrees of freedom, one rotation, and one reflection. No amount of ranging removes those four. Something absolute must supply them, and in this system three different sensors do:

Degree of freedom	What fixes it	Notes
Rotation (1)	Heading measurement	sigma_theta = 0.05 rad; enters the mobility model
Reflection (1)	Heading plus dead reckoning	a consistent trajectory breaks the mirror
Translation (2)	Intermittent GPS	carried between fixes by velocity integration

So the architecture is: ranges fix the shape, heading fixes the orientation, GPS fixes the position, and dead reckoning bridges the gaps. That is a complete and honest account of observability for this system, and it is a better framing for the introduction than "we apply SPAWN".

2. Do not approximate GPS-equipped agents as anchors

The temptation is to treat a drone holding a GPS fix as an anchor so that it stabilises the network. Two reasons not to.

- **It is false confidence.** An anchor has a known position; a GPS fix has a standard deviation of some metres. Declaring it exact injects a claim into the network that is not true, and the messages then propagate that claim to every other node. This manufactures the overconfidence problem described in section 3 rather than avoiding it.
- **It is unnecessary.** A node whose belief is much sharper than its neighbours' already behaves like an anchor with no special casing. Its outgoing message is a thin ring, the product at the receiver is dominated by it, and the information flows correctly on its own. That is what the formalism is for.

Applying GPS as a local likelihood inside the prediction operation — SPAWN's z_self — is the correct treatment and is what the implementation already does. The right question is therefore not "how do I fake an anchor" but "why is the sharp belief not dominating as it should", which section 3 answers.

3. Why more SPAWN iterations can make things worse

A fully connected three-node network is a single triangle. Every edge lies inside a three-cycle, so information leaving node 2 reaches node 1, returns to node 2 on the next iteration, and is multiplied in again as though it were new evidence. Three iterations means three rounds of double counting, and the node with the best information — the one holding GPS — has the most to lose.

SPAWN states the underlying limitation in its own words (Section III-A4, Trees and Cyclic Graphs, Part 2):

> *"The SPA provides us with exact marginals for FGs without cycles (i.e., trees). For FGs with cycles, we can easily extend the SPA, which becomes iterative. However, the computed functions b_Xk and b_Xl are no longer exactly equal to the corresponding marginal a posteriori distributions, and b_X is not necessarily a distribution."*

So degradation with iterations on a cyclic graph is expected behaviour of loopy belief propagation, not an implementation fault. Three practical consequences:

- **Fewer iterations is often better.** Ihler et al. report that 1 to 3 iterations suffice; on a single triangle, 1 may beat 3. This is the cheapest thing to test.
- **Short cycles are the enemy.** Three-cycles return information after two hops. Longer cycles dilute it. Denser is not better.
- **Message damping is the standard remedy** if the dense case must be kept: blend each new message with the previous iteration's, m <- m^alpha * m_old^(1-alpha) with alpha near 0.5. This is general BP practice, not something in SPAWN, so flag it as an implementation choice.

4. Topology: what actually works

For the configuration to be recoverable from ranges alone (up to the global transform of section 1), the network graph must be **generically globally rigid**. A merely rigid graph still admits discrete ambiguities — reflections of part of the formation — which is exactly the mirror-mode behaviour observed with three drones. In the plane the condition is 3-connectivity together with redundant rigidity (Jackson and Jordan, 2005).

The table below was computed directly: the rigidity matrix rank gives rigidity (rank = 2n-3), redundant rigidity is tested by removing each edge in turn, and connectivity by exhaustive vertex removal. R is the ranging radius in units of the nearest-neighbour spacing.

Formation	R	Edges	Degrees	Globally rigid	Tri/edge
Triangle (3)	1.05	3	2-2	NO (ambiguous)	0.33
Line / convoy (5)	any	4	1-2	NO (flexible)	0.00
Diamond (4)	1.80	6	3-3	yes	0.67
V / echelon (5)	1.80	7	2-4	NO (ambiguous)	0.43
Pentagon + centre (6)	1.20	10	3-5	yes	0.50
Hexagon + centre (7)	1.00	12	3-6	yes	0.50
Two staggered rows (6)	1.80	12	3-5	yes	0.83
Grid 3x3 (9), w/ diagonals	1.45	20	3-8	yes	0.80
Random spread (8)	1.80	9	1-3	NO (flexible)	0.11

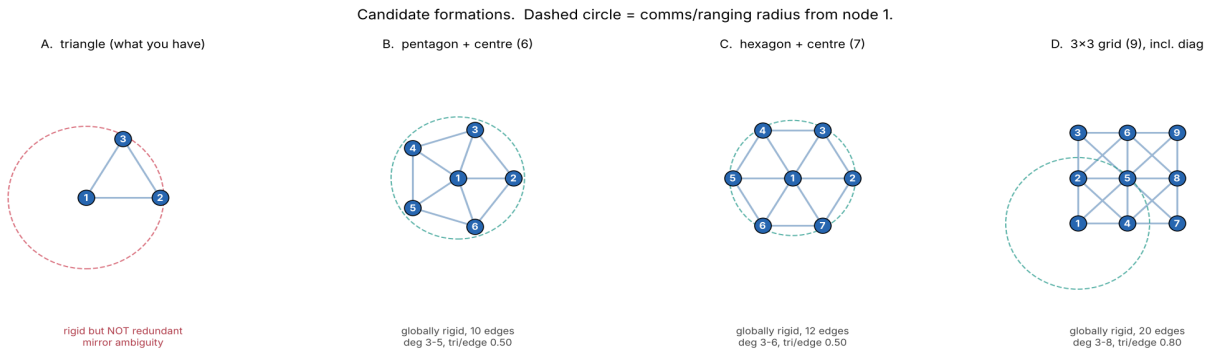


Figure 1. Four candidate formations at their recommended ranging radius. The triangle is rigid but not redundantly rigid, so it admits a mirror solution; the other three are globally rigid.

4.1 The design rule that matters

Sweeping the ranging radius reveals a plateau. The hexagon-plus-centre formation is globally rigid at R = 1.0 to 1.45 with exactly 12 edges and a triangle-to-edge ratio of 0.50. Pushing R to 1.8 adds six edges, buys no additional rigidity, and more than doubles the triangle count to 1.11 per edge. The same pattern holds for every formation tested.

Set the ranging radius just above the rigidity threshold, not as far as the radio can reach. Extra links past that point add nothing to observability and add short cycles, which is precisely what loopy BP handles worst. This is counterintuitive, it is measurable, and it is a defensible design contribution.

4.2 Recommended formations

- **Hexagon with a centre node (7 drones)** — globally rigid at the minimum ranging radius with only 12 edges, minimum degree 3. The most efficient of those tested, and a plausible area-coverage pattern.
- **Pentagon with a centre node (6 drones)** — globally rigid at R = 1.2, 10 edges, degrees 3 to 5. The smallest formation tested that is globally rigid with a clean margin.
- **3x3 grid with diagonals (9 drones)** — the natural lawnmower search pattern. Note that the grid is *not* globally rigid without the diagonals, so the ranging radius must exceed 1.41 times the grid spacing. Worth stating, because it is a real operational constraint rather than an abstraction.
- **Avoid:** the line or convoy (flexible at any radius), the plain V or echelon (ambiguous), and any formation where some node has only two neighbours.

5. Should the topology be time varying?

Yes, and for a reason better than making the algorithm look good. SPAWN is explicitly constructed for time-varying networks: its Net-FG and Net-MP are introduced as "a message schedule that accounts for the time-varying network topology". Implementing a changing topology is therefore faithful to the source, not a flattering special case.

The distinction worth keeping in mind: experiments should be designed to **probe the question**, not to produce a preferred ranking. Choosing a scenario in which cooperation is necessary is legitimate and in fact required — a scenario where every drone has continuous GPS would not test anything. Choosing parameters because they make one method win would not be. Here the two point the same way, so state the reason as the former.

A natural implementation: let drones follow a search pattern that makes neighbours enter and leave ranging radius, and report the resulting graph statistics over time — mean degree, fraction of epochs globally rigid, and the size of the largest connected component. Those three series make a good figure in their own right and give the reader confidence the network was not quietly degenerate.

6. Modelling intermittent GPS credibly

A periodic rule such as "every twentieth step, drone 2 only" looks arbitrary because it is. Three constructions in increasing order of effort, all defensible:

6.1 Two-state Markov availability (recommended default)

Each drone's GPS follows a two-state chain: available to denied with probability p_off, denied to available with p_on. This produces bursty, temporally correlated outages instead of a metronome, and it is parameterised by two interpretable numbers that can be quoted and swept.

```
if gps_state(i)
    if rand < p_off, gps_state(i) = false; end
else
    if rand < p_on,  gps_state(i) = true;  end
end
add_gps_measure = gps_state(i);
```

Mean availability is p_on / (p_on + p_off); mean outage length is 1/p_on steps. Report both in the experiment description.

6.2 A geographic denied zone (recommended for one showcase scenario)

GPS drops when a drone enters a defined region — a canyon, a jamming footprint, terrain multipath. Physically motivated, and it produces a genuinely informative figure: the formation entering the zone, error growing, and recovery on exit. This is the construction that will read as considered rather than arbitrary.

6.3 Varying quality rather than on/off

Real GPS accuracy tracks satellite geometry. Letting the GPS standard deviation vary — say 2 to 8 m as a slow random walk, or tied to a notional visible-satellite count — is more honest than a binary switch and costs

three lines of code.

7. The experiment this unlocks

Reframing the scenario around GPS denial gives a stronger headline result than a particle-versus-Kalman comparison alone. Sweep **GPS availability** as the horizontal axis:

- horizontal axis: expected number of drones holding a fix at any epoch (0, 0.5, 1, 2, 3, ...), or equivalently the mean availability of the Markov model
- two curves: cooperative (SPAWN, drones share ranges) and non-cooperative (each drone dead reckons on its own GPS alone)
- the gap between the curves is the **cooperative gain**, and it answers an operational question: how little absolute information can a swarm get by on if the drones help each other

This is SPAWN's own "benefit of cooperation" result transplanted into a GPS-denied setting. The particle-versus-linearised comparison then sits inside it as the representation question, and the interesting region for that comparison is precisely the low-GPS end, where beliefs are wide and multimodal.

8. References

Verified against the sources on 20 October 2026:

Wymeersch, H., Lien, J. and Win, M. Z. (2009). Cooperative localization in wireless networks. *Proceedings of the IEEE*, 97(2), 427-450. [Algorithm 3; Section III-A4 on cyclic graphs; Section VI for the 500 internode / 2000 intranode sample counts]

Ihler, A. T., Fisher, J. W., Moses, R. L. and Willsky, A. S. (2005). Nonparametric belief propagation for self-localization of sensor networks. *IEEE JSAC*, 23(4), 809-819. [Algorithm 2; Section IV-B on kernel bandwidth; Section VIII for k = 5 oversampling]

Bell, B. M. and Cathey, F. W. (1993). The iterated Kalman filter update as a Gauss-Newton method. *IEEE Transactions on Automatic Control*, 38(2), 294-297.

Long, A., Wolfe, K., Mashner, M. and Chirikjian, G. (2012). The Banana Distribution is Gaussian: A Localization Study with Exponential Coordinates. *Robotics: Science and Systems VIII*, Sydney. DOI 10.15607/RSS.2012.VIII.034

Liu, J. S. and Chen, R. (1998). Sequential Monte Carlo methods for dynamic systems. *JASA*, 93(443), 1032-1044.

Huang, Q. and Leonard, J. J. (2023). GAPSLAM: Blending Gaussian Approximation and Particle Filters for Real-Time Non-Gaussian SLAM. arXiv:2303.14283

Cited from memory — confirm details before submission:

Jackson, B. and Jordan, T. (2005), on generic global rigidity in the plane. Eren, T. et al. (2004), Rigidity, computation, and randomization in network localization, INFOCOM. Aspnes, J. et al. (2006), A theory of network localization, IEEE Trans. Mobile Computing, 5(12). Silverman, B. W. (1986), Density Estimation for Statistics and Data Analysis. Isard, M. (2003), PAMPAS, CVPR. Kong, A., Liu, J. S. and Wong, W. H. (1994), JASA 89(425). Doucet, A. and Johansen, A. M. (2009), A tutorial on particle filtering and smoothing. Gordon, N. J., Salmond, D. J. and Smith, A. F. M. (1993), IEE Proc. F, 140(2). Arulampalam, M. S. et al. (2002), IEEE Trans. Signal Processing, 50(2).